Name : P. Yaswanth

Reg : 192524167

Experiment: Water Jug Problem

Aim

To solve the Water Jug Problem using Python using the State Space Search (Breadth-First Search) algorithm.

Objectives

- To understand the concept of the Water Jug Problem.
- To apply the State Space Search (BFS) algorithm.
- To generate the sequence of steps required to obtain the target amount of water.
- To implement the solution using Python.
- To analyze the efficiency of the search algorithm.

Algorithm

1. Start with both jugs empty (0, 0).
2. Check whether the target amount of water has been reached.
3. Generate all possible next states:
    - Fill Jug 1.
    - Fill Jug 2.
    - Empty Jug 1.
    - Empty Jug 2.
    - Pour water from Jug 1 to Jug 2.
    - Pour water from Jug 2 to Jug 1.
4. Mark visited states to avoid repetition.

5. Continue exploring new states until the target is found.
6. Display the solution path.

Python Program

```
from collections import deque

def water_jug(capacity1, capacity2, target):
    visited = set()
    queue = deque()

    queue.append((0, 0, []))

    while queue:
        jug1, jug2, path = queue.popleft()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        if jug1 == target or jug2 == target:
            print("Solution Path:")
            for state in path:
```

```
            print(state)
        return

    next_states = [
        (capacity1, jug2),  # Fill Jug1
        (jug1, capacity2),  # Fill Jug2
        (0, jug2),          # Empty Jug1
        (jug1, 0),          # Empty Jug2
    ]

    # Pour Jug1 -> Jug2
    transfer = min(jug1, capacity2 - jug2)
    next_states.append((jug1 - transfer, jug2 + transfer))

    # Pour Jug2 -> Jug1
    transfer = min(jug2, capacity1 - jug1)
    next_states.append((jug1 + transfer, jug2 - transfer))

    for state in next_states:
        if state not in visited:
            queue.append((state[0], state[1], path))

print("No solution exists.")
```

Driver Code

water_jug(4, 3, 2)

Sample Output

Solution Path:

(0, 0)

(0, 3)

(3, 0)

(3, 3)

(4, 2)

Out put :

```
from collections import deque

def water_jug(capacity1, capacity2, target):
    visited = set()
    queue = deque()

    queue.append((0, 0, []))

    while queue:
        jug1, jug2, path = queue.popleft()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        if jug1 == target or jug2 == target:
            print("Solution Path:")
            for state in path:
                print(state)
            return

        next_states = [
            (capacity1, jug2),  # Fill Jug1
            (jug1, capacity2),  # Fill Jug2
```

```
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)

=== Code Execution
```

Result

The Water Jug Problem was successfully solved using the State Space Search (Breadth-First Search) algorithm in Python. The program found the sequence of operations required to obtain the target amount of 2 gallons of water.

Conclusion

The experiment demonstrated how the State Space Search (BFS) algorithm can be used to solve the Water Jug Problem efficiently. The program explored all possible states without revisiting the same state, ensuring an optimal solution. This experiment helped in understanding problem-solving techniques in Artificial Intelligence and the practical implementation of search algorithms using Python.